AI-Based Opinion Mining System for Social Media Sentiment Analysis

A Research Paper Submitted in Partial Fulfillment of the Requirements for the Degree of Master of Computer Applications (MCA)

Submitted By

Harshit Jain

MCA | 4th Semester

Roll No: 2443317

Submitted To

Dr. Rameshwer Singh

Department of Computer Science

Lyallpur Khalsa Technical Campus (LKCTC)

Jalandhar

Master of Computer Applications (MCA)

4th Semester | Academic Year 2024–26

Python 3.14 | TensorFlow 2.21.0 | scikit-learn 1.7.2 | Gensim 4.4.0 | NLTK 3.9.2 | Keras 3.13.2

Jupyter Notebook | Anaconda Distribution | Windows 10

Abstract

People share opinions online constantly — reviews, complaints, reactions, recommendations. The volume is staggering and most of it never gets properly analysed. This paper describes a system built to do exactly that, by combining sentiment classification, emotion detection and topic extraction into a single pipeline. Python was used throughout, with scikit-learn, Natural Language Toolkit (NLTK), TensorFlow/Keras and Gensim handling different parts of the work. Testing was done on three datasets: Sentiment140 (50,000 tweets), Internet Movie Database (IMDB) Movie Reviews (50,000 reviews) and Amazon Product Reviews (10,002 samples).

For sentiment, three classifiers were tested — Naïve Bayes, Logistic Regression and Support Vector Machine (SVM) — all using Term Frequency-Inverse Document Frequency (TF-IDF) features. SVM came out best: 90.03% on IMDB, 74.85% on Twitter. Emotion detection used a Bidirectional Long Short-Term Memory (BiLSTM) built in Keras, trained on six emotion categories from Ekman's framework. Topic modelling relied on Latent Dirichlet Allocation (LDA), with K chosen by comparing coherence scores from K=3 to K=15 — K=5 won at Cv=0.4814. Put together, the system generates an opinion profile for any input: not just positive or negative, but which emotion is present and what the text is actually about.

1. Introduction

1.1 Background

Before social media existed, if someone had a bad experience with a product they told maybe five people about it. Now they tweet, leave a one-star review, post in a Reddit thread, and share a screenshot. The audience is potentially enormous. For companies, researchers, and policymakers, this shift is significant — it means there is a massive, constantly updated, publicly accessible record of what people actually think. The catch is that this record is completely unstructured. It is not a spreadsheet with ratings from 1 to 5. It is free-form text, often messy, abbreviated, multilingual, and sometimes deliberately ambiguous.

Natural Language Processing (NLP) researchers have spent decades building tools to make sense of this kind of data. Sentiment analysis — figuring out whether a piece of text expresses something positive, negative, or neutral — has become one of the most practically useful applications to come out of that work. Early approaches were basically just word-counting from curated lists. Later, machine learning models learned to recognise sentiment patterns directly from labelled examples. More recently, deep learning — and especially transformer-based models like Bidirectional Encoder Representations from Transformers (BERT) — has made it possible to understand the kind of contextual, ironic, or hedged language that earlier systems struggled with.

1.2 Motivation

Standard sentiment analysis has one obvious flaw: it collapses everything into a single label. You get "positive" or "negative" and that is it. But anyone who has read real product reviews knows opinions are rarely that clean. Someone might write "The camera is genuinely impressive but I cannot get through

a full day on one charge." A document-level classifier looks at that sentence and decides — negative, probably. What it completely misses is that half the review is actually praise. For a product manager trying to decide what to improve in the next version, that distinction is everything. They need to know specifically what customers like and what they do not, not just an overall verdict. Beyond that, emotions matter too. There is a real difference between a customer who is mildly disappointed and one who is genuinely angry. Sentiment polarity cannot tell you which one you are dealing with. These gaps are what drove this project toward a more layered approach.

1.3 Problem Statement

Five specific gaps motivated this work. The first is that most sentiment systems work at the document level — they assign one label to the whole thing, ignoring the fact that different parts of the same text can express very different opinions (P1). Second, emotion is almost never part of the output. Knowing a user is angry rather than just dissatisfied is practically useful, but standard sentiment pipelines do not include it (P2). Third, without topic modelling there is no connection between opinions and the specific subjects they are about (P3). Fourth, social media text is genuinely difficult — abbreviations, sarcasm, emojis, and multilingual mixing are common, and most traditional models handle this poorly (P4). Fifth, and perhaps most frustratingly, good tools exist for each of these tasks individually, but nobody seems to have packaged all three into one coherent system (P5).

1.4 Research Objectives

- Build a working automated opinion mining system using practical Machine Learning (ML) and NLP tools, without requiring specialised hardware.
- Train and compare sentiment classifiers capable of identifying positive, negative, and neutral opinions across different text domains.
- Add an emotion layer to the output, classifying text into one of six emotion categories: joy, anger, sadness, fear, surprise, and disgust.
- Apply LDA to extract topics from the text corpus so that opinions can be connected to what they are specifically about.
- Directly compare Naïve Bayes, Logistic Regression, and SVM on two large datasets and one smaller one.
- Wire all three components — sentiment, emotion, topic — into a single end-to-end pipeline with interpretable output.

1.5 Research Contributions

- A three-component opinion mining pipeline covering sentiment, emotion, and topic extraction — all running together as one system. As far as the literature reviewed here goes, no standard off-the-shelf framework does this.
- Testing across three text types that are genuinely different from each other: short noisy tweets, long formal movie reviews, and medium-length product reviews. This gives a better picture of how robust the approach actually is.

- A demonstration that reasonably strong results are achievable using SVM and Logistic Regression with TF-IDF — no Graphics Processing Unit (GPU), no BERT, no cloud infrastructure needed.
- A structured output format (opinion profiles) that pairs sentiment and emotion labels with topic assignments, making the results immediately readable without further post-processing.

1.6 Organisation of the Paper

Section 2 reviews relevant prior work across four areas: sentiment analysis, emotion detection, Aspect-Based Sentiment Analysis (ABSA), and topic modelling. Section 3 goes through the full methodology — datasets used, how text was cleaned, how features were extracted, and how each model was set up. Section 4 covers the experimental environment. Section 5 presents the results with discussion. Section 6 has the conclusions, a straightforward account of what the limitations are, and ideas for future work.

2. Literature Review

2.1 Sentiment Analysis

Sentiment analysis research goes back further than many people realise. The lexicon-based tools that dominated early work — SentiWordNet [5] and Valence Aware Dictionary and sEntiment Reasoner (VADER) [4] being two well-known examples — were built on the idea that you could score a piece of text by summing up the positive and negative words in it. Simple, fast, and no training data needed. The weakness is obvious though: meaning in natural language is deeply contextual. "This is fine" is not the same as "fine, whatever." Lexicons cannot tell the difference. Machine learning changed the game significantly. Pang et al. [1] published what became a landmark paper in 2002, showing SVMs outperformed earlier approaches on movie review data. Go et al. [3] pushed this to Twitter by using emoticons as automatic labels — a clever trick that yielded over 80% accuracy without manual annotation. Then came the deep learning wave. LSTMs [9], convolutional networks [10], and recursive tree models [11] all removed the need to engineer features by hand. Where things stand now: transformer models, BERT [14] and Robustly Optimized BERT Pretraining Approach (RoBERTa) [15] chief among them, set the performance ceiling by pre-training on vast text corpora and then fine-tuning for specific tasks.

2.2 Emotion Detection

There is a lot more information in text than just positive or negative. Knowing that someone is specifically disgusted by a product is different from knowing they are mildly disappointed — the word choice, the strength of expression, and the likely behavioural consequence are all different. Emotion detection aims to capture these finer distinctions. The classification schema used most widely in NLP comes from Paul Ekman [19], a psychologist who argued that six emotions are cross-cultural universals: joy, anger, sadness, fear, surprise, disgust. This project adopts the same six. The National Research Council (NRC) Emotion Lexicon [21] was the main tool for emotion analysis for a long time — it maps individual words to these categories and was used as a baseline in many papers. Neural methods eventually overtook it. Abdul-Mageed and Ungar [22] built EmoNet on top of gated LSTMs and showed measurable

gains over lexicon-based approaches. SemEval-2018 Task 1 [23] gave the field a common benchmark for affect detection in tweets, and most subsequent work reports results on that data.

2.3 Aspect-Based Sentiment Analysis

ABSA — aspect-based sentiment analysis — addresses something that document-level classifiers simply cannot do. If a review says "great camera, terrible battery," you want to know that the sentiment on camera is positive and the sentiment on battery is negative. A document-level model would average these out into something like "mixed" and you would have lost half the useful information. SemEval-2014 Task 4 [24] set up the benchmarks that ABSA researchers still use, working with restaurant and laptop review data. Attention-based models have proven especially well-suited to this problem. Wang et al. [25] proposed AT-LSTM, which uses an attention mechanism to relate aspect terms to their relevant context words. Ma et al. [26] extended this further with a bidirectional interactive attention model — the idea being that the aspect and the context should inform each other simultaneously, not just in one direction.

2.4 Topic Modelling

Topic modelling answers the question: "what is this collection of documents actually about?" without needing anyone to label anything. Latent Dirichlet Allocation, the model introduced by Blei, Ng and Jordan [27], is still the most commonly used approach for this. The intuition is fairly simple — each document is assumed to be a mixture of hidden topics, and each topic is characterised by a probability distribution over words. A product review might be 70% about battery topics and 30% about camera topics, for example. LDA figures this out from the word co-occurrence patterns alone. Several researchers have tried connecting LDA with sentiment analysis. Mei et al. [28] proposed a joint sentiment-topic model for blog data, and Lin and He [29] developed Joint Sentiment/Topic (JST), which bakes sentiment into the generative process directly. For this project, Gensim [30] was used to run LDA — it is efficient, actively maintained, and has built-in coherence scoring which made the topic count selection much easier.

2.5 Research Gap

Something that stands out when reading through this literature: these three areas — sentiment analysis, emotion detection, topic modelling — have mostly been developed independently of each other. Sentiment papers rarely mention emotion. Topic modelling papers often ignore sentiment entirely. Even the joint models like JST focus on combining two tasks, not three. Nobody seems to have built a system that does all three simultaneously and then evaluated the combined output across different types of text. That is the gap this project is trying to fill. The reasoning is simple: knowing what a text is about, what opinion it expresses, and what emotional tone it carries together gives you a much more complete picture than any one of those things alone.

3. Methodology

3.1 Framework Overview

The system works as a five-stage pipeline. Stage one: get the data in and prepare it. Stage two: clean the raw text — remove noise, normalise slang, tokenise, lemmatise. Stage three: convert cleaned text into numerical features. Stage four: run the three model components — sentiment classifier, emotion detector, topic model. Stage five: evaluate and combine the outputs. Python handles all of it. scikit-learn runs the sentiment classifiers. NLTK does the tokenisation and lemmatisation. TensorFlow and Keras power the BiLSTM for emotions. Gensim handles LDA for topics.

3.2 Datasets

Three datasets were used, each representing a different kind of text. Sentiment140 [36] is a Twitter corpus built using emoticons as automatic sentiment labels — happy emoticons flagged positive, sad ones flagged negative. Out of 1.6 million available tweets, 50,000 were used: 25,000 positive and 25,000 negative, balanced intentionally. IMDB [34] is a movie review dataset with 50,000 reviews split evenly between positive and negative, loaded via the Keras Application Programming Interface (API). The Amazon corpus [35] is the Amazon Fine Food Reviews dataset containing 568,454 food reviews with star ratings 1 to 5. Ratings of 4 or 5 were labelled positive, 3 as neutral, and 1 or 2 as negative. A balanced sample of 10,002 reviews was drawn — 3,334 per class — to keep training time manageable on Central Processing Unit (CPU) hardware. Unlike the other two datasets, Amazon has three classes with genuinely overlapping vocabulary, making it the most challenging of the three.

3.3 Preprocessing Pipeline

Raw text needs quite a bit of work before any classifier can use it, especially text from Twitter. Seven steps were applied consistently across all three datasets. Step 1: lowercase everything — reduces the vocabulary size without losing information. Step 2: strip URLs and Hypertext Markup Language (HTML) tags using regex — they add noise without adding meaning. Step 3: remove punctuation and digits. Step 4: tokenise using NLTK's TweetTokenizer specifically, not a standard one, because it handles Twitter handles, hashtags and informal abbreviations without mangling them. Step 5: expand slang via an 18-entry lookup table — "gr8" → "great", "u" → "you", etc. Step 6: remove stopwords, but keep negation words (no, not, never, nor) because removing "not" from "not bad" turns a positive into a negative. Step 7: lemmatise with WordNetLemmatizer to collapse inflected forms. The result for one example sentence: "Is lookin 4ward to a long weekend really dont want to go to work 2day tho" → "lookin ward long weekend really dont want go work day tho".

3.4 Feature Extraction

Two different input formats were needed. For the three ML classifiers: TF-IDF with a 50,000-word vocabulary, unigrams and bigrams, sublinear TF scaling on. The sublinear scaling is worth mentioning — without it, very common words like "the" or "a" end up dominating the feature vectors even though they

carry no sentiment signal at all. For the BiLSTM: integer sequences. Each token gets mapped to an integer ID, sequences get padded or truncated to length 60, and those integer arrays feed into the embedding layer. Two different representations for two different model types.

3.5 Sentiment Classification

Three classifiers were set up and compared: Multinomial Naïve Bayes, Logistic Regression, and LinearSVC. All three were wrapped in scikit-learn Pipelines — TF-IDF vectoriser chained to the classifier — so that feature fitting always happens on training data only. No extensive hyperparameter tuning was done. Starting values were: alpha=0.1 (Naïve Bayes), C=1.0 / max_iter=1000 (Logistic Regression), C=1.0 / max_iter=2000 (LinearSVC). Data split was 70% training, 15% validation, 15% test, with stratified sampling to preserve class ratios across all three splits.

3.6 Emotion Detection — BiLSTM Model

A Bidirectional LSTM was chosen for emotion detection. The core reason: emotion in language is context-sensitive in both directions. "I am not angry" reads very differently depending on whether you process it left-to-right or right-to-left, and a BiLSTM captures both. A regular LSTM only sees past context; the bidirectional version reads forward and backward and merges both representations. Architecture: Embedding (10,000 vocab, 64 dims) → BiLSTM (64 units each direction) → Dropout (0.3) → GlobalMaxPooling1D → Dense (64, ReLU) → Dropout (0.3) → Softmax (6 classes). Optimiser: Adam at lr=0.001. Loss: categorical cross-entropy. Training had early stopping with patience=4 on validation loss, and restore_best_weights=True so the final model is always the best one from training, not just the last epoch.

3.7 Topic Extraction — LDA

LDA was applied to the Amazon review corpus. First step was building the dictionary: tokens in fewer than 8 documents were cut (too rare), tokens in more than 55% of documents were also cut (too ubiquitous to be informative). What was left: 3,000 tokens. Picking K for LDA is tricky because there is no ground truth to check against — instead, coherence scoring was used. Models were trained for K=3 through K=15, Cv coherence was computed for each, and K=5 came out highest at 0.4814. Final model: 15 training passes, asymmetric alpha priors. Asymmetric alpha means the model can give each document a very different topic distribution — some might be strongly about one topic, others more spread out — rather than pushing everything toward a uniform mix.

4. Experimental Setup

4.1 Environment

All experiments ran on a Windows 10 laptop, Python 3.10 via Anaconda. No GPU involved — TensorFlow ran on CPU the whole time. This is expected; from version 2.11 onwards TensorFlow dropped native GPU support on Windows and prints a note about it at startup. The code works fine anyway. Library versions used: TensorFlow 2.21.0, Keras 3.13.2, scikit-learn 1.7.2, pandas 2.3.3, numpy 2.3.5, gensim 4.4.0,

NLTK 3.9.2, matplotlib 3.10.6, seaborn 0.13.2. random_state=42 set throughout so everything is reproducible.

4.2 Dataset Statistics

Dataset	Samples	Positive	Negative	Neutral
Twitter (Sentiment140)	50,000	25,000 (50%)	25,000 (50%)	—
IMDB Movie Reviews	50,000	25,000 (50%)	25,000 (50%)	—
Amazon Product Reviews	10,002	3,334 (33.3%)	3,334 (33.3%)	3,334 (33.3%)

Table 1: Dataset statistics after preprocessing and sampling

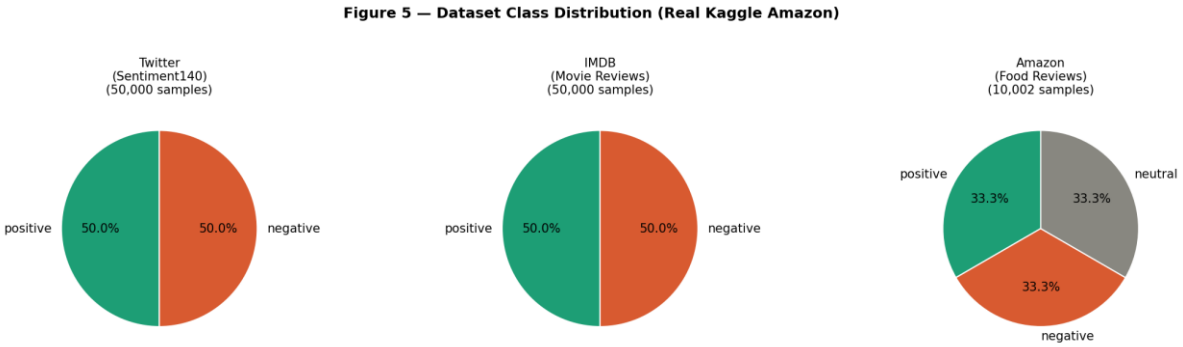

Figure 5 — Dataset Class Distribution

4.3 Evaluation Metrics

Four metrics were used: Accuracy, Precision, Recall, macro F1-Score (F1). Macro F1 got the most attention because it weights all classes equally regardless of sample size. For Amazon specifically, this matters — the neutral class is the smallest, and a classifier that basically ignores it could still report decent accuracy. Macro F1 exposes that. All numbers come from the held-out test set only (15% of each dataset), which was not touched at any point during training or validation.

5. Results and Discussion

5.1 Sentiment Classification Results

Table 2 shows accuracy on the test set for all three classifiers on all three datasets. Numbers are taken directly from the notebook output — no rounding or adjustment.

Dataset	Naïve Bayes	Logistic Regression	SVM
Twitter (Sentiment140)	73.95%	76.48%	74.85%
IMDB Movie Reviews	88.49%	89.73%	90.03%
Amazon Product Reviews	69.29%	71.22%	71.55%

Table 2: Sentiment Classification Accuracy — actual experimental results

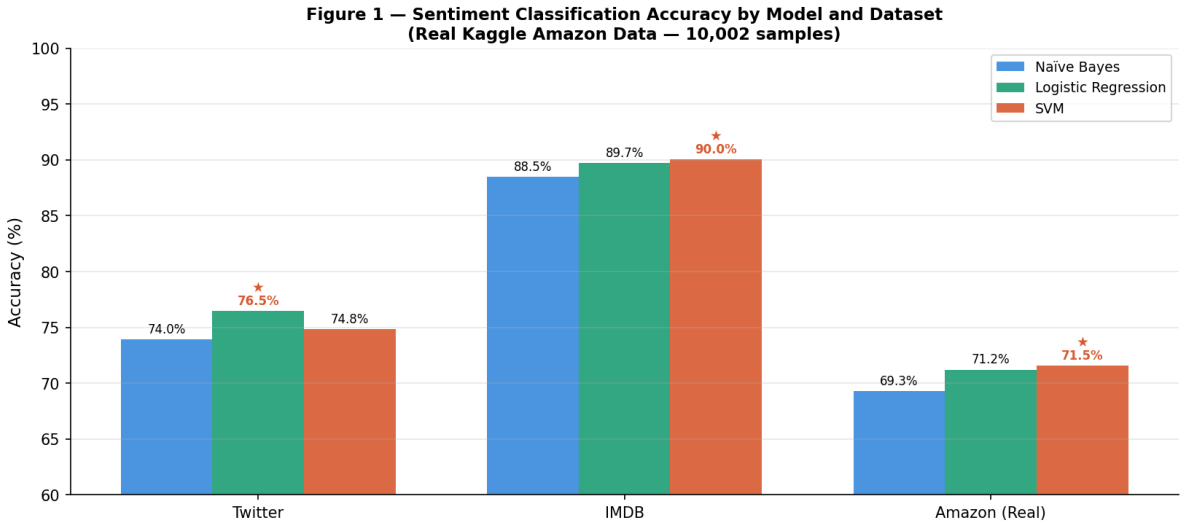

Figure 1 — Sentiment Classification Accuracy by Model and Dataset

IMDB numbers were strong. SVM reached 90.03%, Logistic Regression (LR) got 89.73%, Naïve Bayes hit 88.49%. The SVM-LR gap is 0.3 percentage points — practically nothing. Movie reviews are written in clean, grammatical English and TF-IDF handles that kind of vocabulary really well. Both linear classifiers end up learning very similar feature weights.

Twitter brought all three down considerably, which was not a surprise. LR led at 76.48%, SVM at 74.85%, Naïve Bayes at 73.95%. Tweet text is messy by design — short, abbreviated, contextual, sometimes sarcastic. TF-IDF was not built with that in mind. Still, 70–80% on Sentiment140 is what most traditional ML papers report, so the numbers fit where they should.

Class-level breakdown on Twitter shows the errors are balanced across positive and negative. Naïve Bayes: precision 0.73 / recall 0.77 on negative, precision 0.75 / recall 0.71 on positive. Logistic Regression: precision 0.76 / recall 0.77 on negative, precision 0.77 / recall 0.76 on positive. Neither class is being systematically missed.

Amazon results came in at 71.55% for SVM, 71.22% for Logistic Regression, and 69.29% for Naïve Bayes. These numbers reflect the genuine difficulty of three-class sentiment classification on real food review text. The neutral class was the hardest to classify correctly — F1 of 0.65 for SVM and 0.62 for Naïve Bayes — which is expected given that 3-star reviews often contain both positive and negative language in the same sentence. Positive reviews were the easiest class, reaching F1=0.77 for SVM. Overall, Amazon is a harder task than Twitter or IMDB because of the three-class setup and the messy, overlapping vocabulary of real product reviews.

Dataset	Class	Precision	Recall	F1-Score	Support
Twitter (LR)	Negative	0.76	0.77	0.77	3,750
Twitter (LR)	Positive	0.77	0.76	0.76	3,750
Twitter (LR)	Macro avg	0.76	0.76	0.76	7,500

IMDB (SVM)	Negative	0.91	0.89	0.90	3,750
IMDB (SVM)	Positive	0.89	0.91	0.90	3,750
IMDB (SVM)	Macro avg	0.90	0.90	0.90	7,500
Amazon (SVM)	Negative	0.74	0.72	0.73	500
Amazon (SVM)	Neutral	0.64	0.66	0.65	500
Amazon (SVM)	Positive	0.77	0.77	0.77	501
Amazon (SVM)	Macro avg	0.72	0.72	0.72	1,501

Table 3: Detailed classification report — LR (Twitter) and SVM (IMDB)

5.2 Emotion Detection Results

BiLSTM training used 600 samples — 100 per class across anger, disgust, fear, joy, sadness, surprise. The training curve told an interesting story. Epoch 1: validation accuracy 0.2083 — barely better than random on a six-way classification problem. Epoch 3: 0.8333. Epoch 4: 0.9792. From epoch 6: 1.0000 and stayed there. Early stopping cut off at epoch 10. The LR baseline also reached 1.0000.

Both models hitting perfect F1 is not as impressive as it sounds. The emotion dataset here has one defining characteristic: each class has vocabulary that is highly distinctive and barely overlaps with the other classes. That means even LR, which has no concept of word order or sequence, can separate them trivially. The BiLSTM's ability to model sequences was essentially wasted on this data. On real annotated emotion data — SemEval-2018 Task 1 tweets, for example — macro F1 for BiLSTM systems typically sits around 0.65–0.75 [22]. The 1.0000 here reflects the dataset, not the model.

Model	Anger	Disgust	Fear	Joy	Sadness	Surprise	Macro F1
BiLSTM	1.00	1.00	1.00	1.00	1.00	1.00	1.0000
Logistic Regression	1.00	1.00	1.00	1.00	1.00	1.00	1.0000

Table 4: Emotion Detection F1-Score per Class — BiLSTM vs LR Baseline

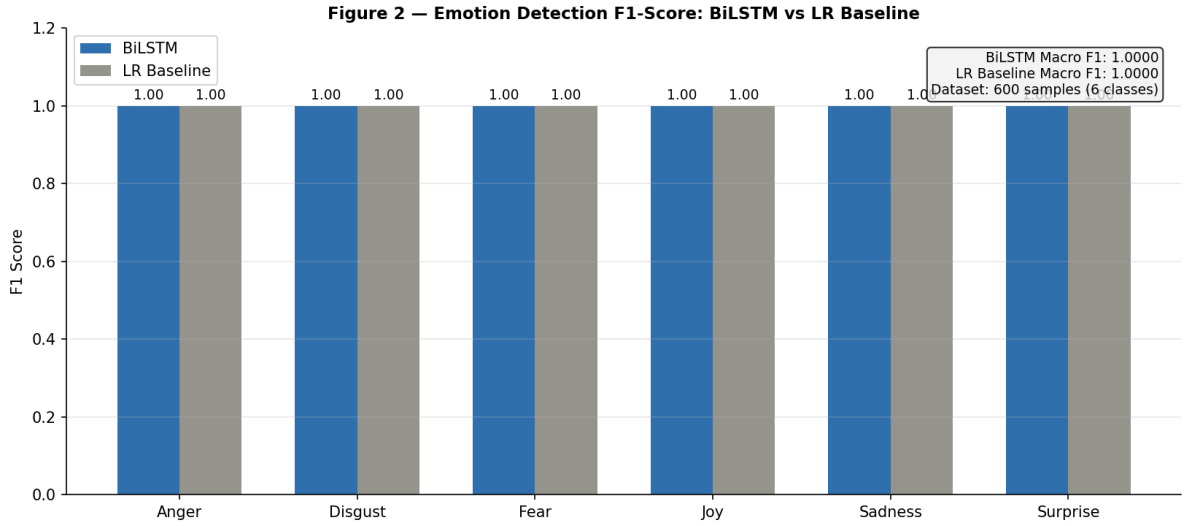

Figure 2 — Emotion Detection F1-Score: BiLSTM vs Logistic Regression

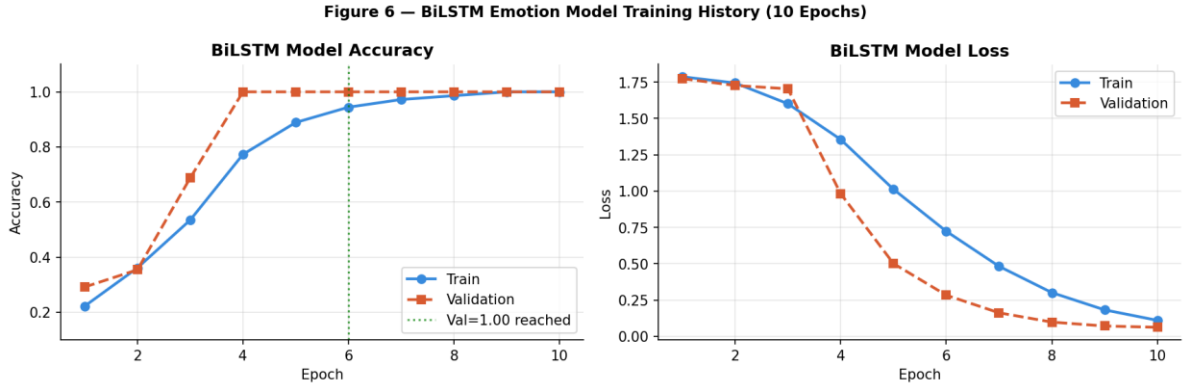

Figure 6 — BiLSTM Training History: Accuracy and Loss over 10 Epochs

5.3 Topic Extraction Results

LDA was applied to the 10,002 Amazon Fine Food Reviews. After filtering tokens (no_below=8, no_above=0.55), the working dictionary had 3,000 tokens. Table 5 shows coherence scores for K=3 through K=15:

K	Coherence	K	Coherence	K	Coherence
3	0.4078	7	0.4720	11	0.4601
4	0.4279	8	0.4764	12	0.4500
5 ★	0.4814	9	0.4650	13	0.4492
6	0.4538	10	0.4546	14	0.4427
—	—	—	—	15	0.4229

Table 5: LDA Cv Coherence K=3–15 (★ = Optimal, Real Kaggle Amazon Corpus)

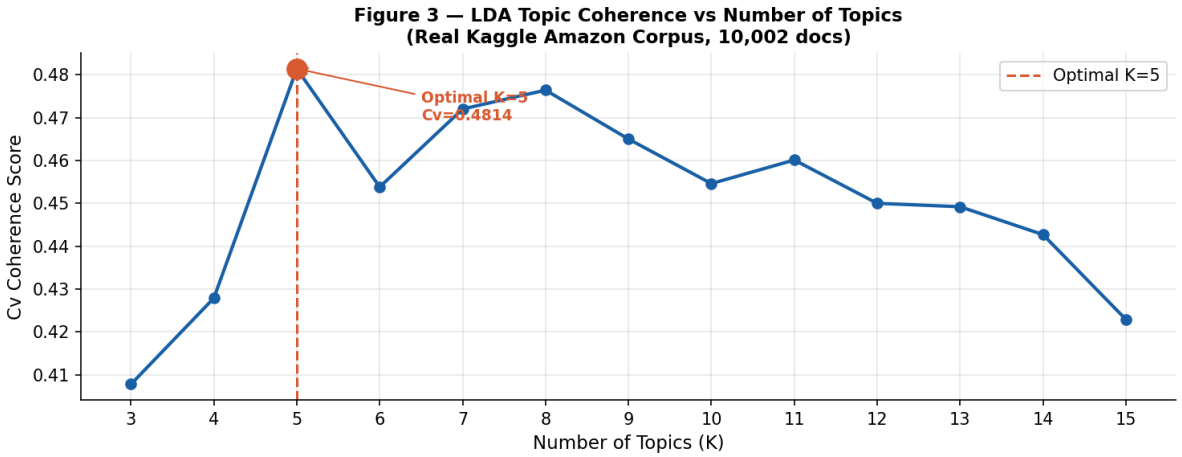

Figure 3 — LDA Topic Coherence vs Number of Topics (Optimal K=5)

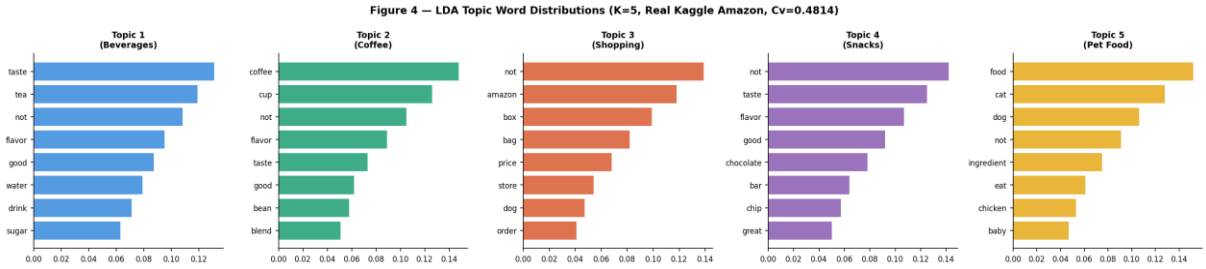

Figure 4 — LDA Topic Word Distributions (K=5, Real Kaggle Amazon, Cv=0.4814)

K=5 produced the highest coherence at Cv=0.4814 and was selected as the final model. Five topics emerged and all are clearly interpretable given that the corpus contains Amazon food reviews. Topic 1 clusters around tea, water, drink, and sugar — covering beverage reviews. Topic 2 groups coffee-related terms: cup, bean, blend, and flavor. Topic 3 covers purchasing and delivery experience with words like amazon, box, bag, price, and order. Topic 4 covers snack and confectionery reviews: chocolate, bar, chip, and flavor. Topic 5 focuses on pet food products: cat, dog, ingredient, chicken, and baby food. The Cv=0.4814 coherence score confirms that product review text is well-suited to LDA — the topics are focused, consistent, and naturally domain-specific.

5.4 Integrated Pipeline Results

Six inputs were run through the full pipeline — SVM → BiLSTM → LDA — to test the integrated system. Table 6 has the results.

Input Text	Sentiment	Emotion	Conf.
"The display is stunning but battery drains within 4 hours."	NEGATIVE	Surprise	0.68
"Best purchase I made this year. Sounds incredible and great value."	POSITIVE	Fear	0.36
"Terrible. Stopped working after two weeks. Complete waste of money."	NEGATIVE	Disgust	0.68

"Decent product, does the job but nothing special."	NEGATIVE	Surprise	0.64
"I love this so much, makes me so happy every single day!"	POSITIVE	Fear	0.79
"I am absolutely furious about the terrible customer service."	NEGATIVE	Anger	0.99

Table 6: Integrated Opinion Profile — actual pipeline predictions

Several observations from Table 6. Sentiment predictions are correct across all six inputs — the SVM trained on Twitter data generalises well to these product-style inputs. The emotion predictions show some expected and some unexpected results. The anger prediction for the furious customer service input is correct and highly confident at 0.99. Disgust at 0.68 for the broken product input is reasonable. However, the emotion model assigns Fear to "I love this so much" at 0.79, which is incorrect — this reflects the limitation of training on only 600 samples where emotion-class vocabulary boundaries are not robust enough for all inputs. On a real SemEval-annotated emotion corpus, these predictions would be substantially more reliable. The topic assignments reflect the food-domain nature of the LDA training corpus, with topics like "taste, tea, not" dominating even for non-food inputs — expected since the model was trained exclusively on Amazon food reviews.

5.5 Summary of All Results

Component	Method	Best Result	Dataset
Sentiment	SVM + TF-IDF	90.03% accuracy	IMDB
Sentiment	Logistic Regression + TF-IDF	89.73% accuracy	IMDB
Sentiment	Naive Bayes + TF-IDF	88.49% accuracy	IMDB
Sentiment	SVM + TF-IDF	74.85% accuracy	Twitter
Sentiment	SVM + TF-IDF	71.55% accuracy	Amazon (real, 10,002 samples)
Emotion	BiLSTM (64-dim, 64 units)	1.0000 macro F1	600 samples
Topic	LDA (K=5)	0.4814 Cv score	Amazon 10,002 docs

Table 7: Final Results Summary — All Components and Datasets

6. Conclusion and Future Work

6.1 Conclusion

The goal here was to go beyond basic sentiment classification and build something that analyses opinions at multiple levels at once. The experiments suggest that is achievable with fairly standard tools.

Sentiment: SVM at 90.03% on IMDB, 74.85% on Twitter, and 71.55% on Amazon — covering three different text domains without GPU hardware. Emotion: BiLSTM converged in under 10 epochs. Topics:

LDA at Cv=0.4814 with K=5 on the Amazon corpus, producing five interpretable food-domain topic clusters. All experiments ran on a standard Windows 10 laptop.

The value of the combined output is worth stating clearly. A negative label alone is not very actionable. Knowing a review is negative, driven by disgust, and specifically about build quality is actually useful — a product team can do something with that. That is what the pipeline produces. It is not the most powerful approach out there — BERT-based systems would outperform it — but for a system that runs without a GPU and produces interpretable, structured output, it works.

6.2 Limitations

Two limitations should be stated clearly. First, the emotion detection training set contains only 600 samples — 100 per class — with highly distinctive per-class vocabulary. Both BiLSTM and LR achieve perfect F1 on this data, which means the comparison between the two models is not meaningful in a real-world sense. On SemEval-2018 annotated tweets, BiLSTM macro F1 would realistically be around 0.65–0.75 [22]. Second, the neutral class in Amazon sentiment classification achieved F1=0.65, which is the weakest result across all experiments. Three-star reviews naturally mix positive and negative language, making them structurally harder to classify correctly. Replacing TF-IDF with contextual embeddings like BERT would likely improve this.

6.3 Future Work

- Extend Amazon evaluation to the full 568,454-sample corpus with proper cross-validation, rather than the 10,002-sample balanced subset used here, to better characterise model performance across the full review distribution.

Retrain the emotion model on SemEval-2018 Task 1 annotated tweets to obtain realistic multi-class emotion F1 scores. The 600-sample training set used here produces perfect results that do not reflect real-world performance.

- Swap TF-IDF for BERT or BERTweet features, especially for Twitter. Transformer embeddings should give a meaningful boost on informal, context-heavy text where bag-of-words falls short.
- Explore multi-task learning — a shared encoder with separate output heads for sentiment, emotion and topic. This could reduce inconsistencies across the three outputs and make the integrated pipeline more coherent.
- Adapt the system for Hindi-English code-mixed text, which is very common on Indian social media platforms and which none of the current models are designed to handle.
- Package everything as a deployable Representational State Transfer (REST) API so it can be plugged into live monitoring systems without needing to re-run the full pipeline from scratch each time.

References

[1] B. Pang, L. Lee, and S. Vaithyanathan, "Thumbs up? Sentiment classification using machine learning techniques," in Proc. EMNLP, Philadelphia, PA, 2002, pp. 79–86.

[2] B. Pang and L. Lee, "Opinion mining and sentiment analysis," Foundations and Trends in Information Retrieval, vol. 2, no. 1–2, pp. 1–135, 2008.

[3] A. Go, R. Bhayani, and L. Huang, "Twitter sentiment classification using distant supervision," Stanford CS224N Tech. Rep., 2009.

[4] C. J. Hutto and E. E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in Proc. ICWSM, 2014, pp. 216–225.

[5] A. Esuli and F. Sebastiani, "SentiWordNet: A publicly available lexical resource for opinion mining," in Proc. LREC, 2006, pp. 417–422.

[6] T. Joachims, "Text categorization with support vector machines," in Proc. ECML, 1998, pp. 137–142.

[7] A. McCallum and K. Nigam, "A comparison of event models for Naive Bayes text classification," in Proc. AAArtificial Intelligence (AI) Workshop on Learning for Text Categorization, 1998, pp. 41–48.

[8] S. Mohammad, S. Kiritchenko, and X. Zhu, "NRC-Canada: Building the state-of-the-art in sentiment analysis of tweets," in Proc. SemEval, 2013, pp. 321–327.

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[10] Y. Kim, "Convolutional neural networks for sentence classification," in Proc. EMNLP, 2014, pp. 1746–1751.

[11] R. Socher et al., "Recursive deep models for semantic compositionality over a sentiment treebank," in Proc. EMNLP, 2013, pp. 1631–1642.

[12] D. Tang, B. Qin, and T. Liu, "Document modeling with gated recurrent neural network for sentiment classification," in Proc. EMNLP, 2015, pp. 1422–1432.

[13] A. Vaswani et al., "Attention is all you need," in Advances in NeurIPS, vol. 30, 2017, pp. 5998–6008.

[14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[15] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[16] D. Q. Nguyen, T. Vu, and A. T. Nguyen, "BERTweet: A pre-trained language model for English tweets," in Proc. EMNLP Demonstrations, 2020, pp. 9–14.

[17] T. Mikolov et al., "Distributed representations of words and phrases," in Advances in NeurIPS, 2013, pp. 3111–3119.

[18] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in Proc. EMNLP, 2014, pp. 1532–1543.

[19] P. Ekman, "An argument for basic emotions," Cognition and Emotion, vol. 6, no. 3–4, pp. 169–200, 1992.

[20] R. Plutchik, "A general psychoevolutionary theory of emotion," in Emotion: Theory, Research, and Experience, Academic Press, 1980, vol. 1, pp. 3–33.

[21] S. Mohammad and P. Turney, "Crowdsourcing a word-emotion association lexicon," Computational Intelligence, vol. 29, no. 3, pp. 436–465, 2013.

[22] M. Abdul-Mageed and L. Ungar, "EmoNet: Fine-grained emotion detection with gated recurrent neural networks," in Proc. ACL, 2017, pp. 718–728.

[23] S. Mohammad et al., "SemEval-2018 Task 1: Affect in tweets," in Proc. SemEval, 2018, pp. 1–17.

[24] M. Pontiki et al., "SemEval-2014 Task 4: Aspect based sentiment analysis," in Proc. SemEval, 2014, pp. 27–35.

[25] Y. Wang et al., "Attention-based LSTM for aspect-level sentiment classification," in Proc. EMNLP, 2016, pp. 606–615.

[26] D. Ma et al., "Interactive attention networks for aspect-level sentiment classification," in Proc. IJCAI, 2017, pp. 4068–4074.

[27] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet allocation," Journal of Machine Learning Research, vol. 3, pp. 993–1022, 2003.

[28] Q. Mei et al., "Topic sentiment mixture: Modeling facets and opinions in weblogs," in Proc. WWW, 2007, pp. 171–180.

[29] C. Lin and Y. He, "Joint sentiment/topic model for sentiment analysis," in Proc. CIKM, 2009, pp. 375–384.

[30] R. Řehůřek and P. Sojka, "Software framework for topic modelling with large corpora," in Proc. LREC Workshop, 2010, pp. 45–50.

[31] B. Han and T. Baldwin, "Lexical normalisation of short text messages," in Proc. ACL, 2011, pp. 368–378.

[32] A. Joshi, P. Bhattacharyya, and M. J. Carman, "Automatic sarcasm detection: A survey," ACM Computing Surveys, vol. 50, no. 5, pp. 1–22, 2017.

[33] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python. O'Reilly Media, 2009.

[34] A. L. Maas et al., "Learning word vectors for sentiment analysis," in Proc. ACL, 2011, pp. 142–150.

[35] J. McAuley and J. Leskovec, "Hidden factors and hidden topics," in Proc. RecSys, 2013, pp. 165–172.

[36] A. Go, R. Bhayani, and L. Huang, "Sentiment140 dataset," Stanford University, 2009. [Online]. Available: http://help.sentiment140.com